CROSS / PROBLEM 03

Problem 3: Faster encrypted LeNet and AlexNetTiny

CADathlon · `agent_pset` · AI agents allowed · BONUS problem · 20 points · proposed duration: 8 hours

1. Introduction

An encrypted classifier protects an image while a remote server evaluates a neural network. CROSS already implements LeNet and AlexNetTiny using CKKS. Your task is to improve **the latency and throughput of both encrypted networks** while preserving their outputs, weights and cryptographic security.

AI agents are allowed. You may use coding agents and AI assistants to design, implement, debug, test and profile your solution. Disclose the tools and assisted work in `REPORT.md`, and independently verify generated code and results. All correctness, privacy, security and scoring requirements apply.

This is a compiler and accelerator optimization problem. The trained models are fixed. BAT turns suitable high-precision arithmetic into low-precision matrix operations, while MAT reduces runtime data rearrangement. Use profiles to identify when arithmetic, rotations, packing, compilation or memory traffic limits the current CROSS path.

2. Problem statement

2.1 Two required networks

Network	Exact encrypted architecture	Data and client preprocessing
LeNet (`QuadLeNet`)	Conv 1→4, 5×5, stride 2, padding 2; square; Conv 4→8 with the same kernel/stride/padding; square; flatten 392; FC 392→32; square; FC 32→10	MNIST, 1×28×28; convert uint8 to [0,1], then `(x-0.1307)/0.3081`
AlexNetTiny (`QuadAlexNetTinyInfer`)	Conv 3→4, 3×3, pad 1; average pool 2; square; Conv 4→8, 3×3, pad 1; average pool 2; square; Conv 8→16, 3×3, pad 1; adaptive average pool to 2×2; square; flatten 64; FC 64→10	CIFAR-10, 3×32×32; normalize channels with means (0.4914,0.4822,0.4465) and stds (0.2023,0.1994,0.2010), then client-side 2×2 average pool to 3×16×16

Use the exported BatchNorm-folded AlexNetTiny inference weights. BatchNorm must not be reapplied. All preprocessing is fixed and occurs on the client before encryption. The server receives only encrypted preprocessed images. The client decrypts ten logits and chooses argmax. Model weights are plaintext server constants; the privacy target is the client's image.

Organizers supply frozen, trained weights and hash manifests for both models. The repository's `LeNetDemo` and `AlexNetTinyDemo` default to seeded-random weights; those are useful for arithmetic tests but do not establish classifier accuracy. Load **every** parameter from the official archive before the ranked run. Public smoke tests deliberately use random weights and are labelled so.

2.2 What to implement

Implement `build_mapping(model, input_shape, *, batch, devices)` in `src/cnn_opt.py`. Return a prepared CROSS `Mapping` (or a compatible object with `encrypt_input`, `execute`, `decrypt_output`, `release`). The supplied model is an ordinary evaluation-mode `torch.nn.Module` with fixed weights. The baseline follows:

```
program = nn.vectorize(model, input_shape)
packed = packing.pack(program, packing.PackingPolicy(lazy_constants=True))
mapping = Mapping(packed, global_batch=batch, devices=devices)
```

Preserve this model-serving boundary. `execute` accepts/returns a canonical rank-5 `Polynomial`; it cannot decrypt, call the plaintext model, inspect labels, cache results by image ID or return precomputed answers. The standard Mapping owns a context that also supports the client codec; the local runner is a trusted test harness, not an isolated production server. Evaluator code must not use that secret-key access. Include a source-level boundary audit.

You may alter packing templates, plaintext-constant representation, BSGS planning, rotation reuse, kernel fusion, scheduling, data placement, memory reuse and exact modular kernels. Use the same optimization for arbitrary weights and inputs of the prescribed shapes. Edits to CROSS belong in `cross.patch`; edits under this problem's `src/` belong in the submission. You may add helpers. Do not change the benchmark, scoring, fixtures or checks.

You may not change weights, activations, input resolution, numerical preprocessing, network topology, test data, batch sizes or assigned devices. No pruning, retraining, approximate model replacement or plaintext fallback. Keep the organizer baseline's validated RingConfig, Q/P chains, scale policy, key-switch decomposition and 128-bit security target. The currently derived plans for both networks have depth 7, degree 32768, 16384 slots, 8 logical Q moduli and dnum 4. The release manifest and actual plan are authoritative; print and compare them before timing. Small insecure test rings receive no performance credit. Secure encryption noise must remain freshly sampled.

Complete these tasks:

1. **P2.1 — establish the baseline:** verify model/parameter provenance, run the exact cleartext and HE paths, and profile each model at batch 1.
2. **P2.2 — improve LeNet:** optimize at least one measured bottleneck and explain which operations or memory transfers it reduces.
3. **P2.3 — improve AlexNetTiny:** measure whether the same change helps; add a model-independent improvement if necessary.
4. **P2.4 — compare and explain:** run all six workload configurations, preserve numerical and label accuracy, and report speedup with evidence.

Suggested starting points: lazy constants; avoiding repeated setup in serving; BSGS diagonal structure and rotation-key reuse; repack elimination; fusion; memory lifetimes. CROSS chooses BSGS plans inside Mapping; obsolete per-demo `bsgs_ratio` arguments and saved network caches are not valid tuning APIs. Use `mapping.estimate_live_memory()` for a logical estimate, and a profiler for actual peak HBM. Clearly distinguish the two measurements.

2.3 Inputs, outputs and six ranked workloads

The public `test1.in` – `test6.in` are JSON descriptions with model, batch size, synthetic data seed and weight seed. `testN.out` contains real cleartext logits for every sample. The runner initializes a complete deterministic weight set and checks shape, finite values, logit error and argmax agreement after HE. These fixtures use batches 1, 2 and 4 on a single device to support local development; they are not the full ranked image-accuracy suite.

All six ranked cases use one identical TPU v6e chip:

Case	Model	Global batch	Dataset
C1	LeNet	1	MNIST
C2	LeNet	8	MNIST
C3	LeNet	32	MNIST
C4	AlexNetTiny	1	CIFAR-10
C5	AlexNetTiny	8	CIFAR-10
C6	AlexNetTiny	32	CIFAR-10

Organizers provide 64 public calibration images and select 256 disjoint hidden test images per model, with 25 or 26 examples per class. Identical images and weights are used for the baseline and each candidate. Evaluate all 256 images per batch configuration, partitioned into full batches without padding or dropping samples. Public boundary cases additionally include all zeros, uniform values, high-contrast pixels and distinct images in every batch lane. The harness may change seeds and weights for structural tests to catch specialization; ranked accuracy always uses the frozen trained weights.

For **each** case all logits must satisfy `abs(HE-clear) <= 0.02 + 0.01*abs(clear)`, at least **99%** of image argmaxes must match the cleartext reference, and label accuracy may fall by at most **1 percentage point** relative to that same model's cleartext predictions on the same images. Report absolute accuracy too. Require all three gates; argmax agreement alone could conceal a broken classifier. NaN/Inf, a shape mismatch or dropped images fails the case. Organizers validate that the unmodified baseline passes before release. Tolerances are contest choices, not guarantees from CKKS.

3. How to compile and run

The organizer image contains Python, NumPy, PyTorch, JAX, the TPU runtime, and the pinned CROSS checkout. `CROSS_ROOT` selects another checkout if needed. The six public reference tests run without training or downloading.

```
cd cadathlon/agent_pset/p3_cnn/src
make all
make reference-test      # numerical fixtures only, no encryption
make baseline-plan       # inspect both actual CROSS plans without keygen
make baseline-test       # unmodified encrypted CROSS, test1; minutes/GB
make test                # all six cases using your cnn_opt.py implementation
python runner.py --case ../test/test4.in --baseline
make submit              # writes ../submission.tar.gz
```

The editable TODO deliberately raises `NotImplementedError` until completed. `baseline.py` provides working reference code; copying it produces a valid starting point with no speedup. Running `make all` only checks Python syntax. Logs go to stderr and one machine-readable JSON result goes to stdout per case. Do not byte-compare noisy CKKS outputs to `.out`; use the stated bounds.

Runtime limits: one TPU v6e chip, the same fixed host image for all teams, 128 GiB maximum host RSS, and no device OOM. Prepare/keygen/compile may take at most **30 minutes per model/batch configuration**. Each online batch may take at most **120 seconds**. A failed or timed-out case scores zero. CPU runs are useful for correctness, but do not count toward the TPU performance score.

Prepare once per configuration. Release it before building the next one. Perform two untimed warm-up requests, then measure five runs on distinct batches; use a fixed published selection from the 256 images for timing. Fresh encryption is required each time. Synchronize output ciphertexts with `jax.block_until_ready` before stopping the server timer. Count dispatch, server computation, online host work and mandatory transfers; no lazy work may escape timing. Report median and p95 (nearest-rank definition) of the five samples. Ranked latency `T` is the median **end-to-end batch wall time** from client encryption through decryption. Also report server-only timing, `T / B` seconds per image and `B / T` images per second. Baseline and candidate use identical boundaries, images, ring, batching and device counts.

Compilation, key generation and constant preparation are reported separately and excluded from warm `T`. They still obey the setup limit. Reuse only public weights/constants, keys and executables, not image-specific ciphertext intermediates. The judge runs baseline and candidate in separate clean processes, interleaves their reservations and reruns ties within 2% once. Record physical peak device memory and process RSS, plus logical memory estimates if used. A throughput increase obtained by changing the batch or hardware does not count as an optimization.

4. How to score

This is the **bonus problem** of the benchmark/CNN track. It is worth **20 points on top of** the 100 available in Problem 1, which is the essential problem. Attempt this one only once Problem 1 is done; a strong result here does not compensate for a weak one there.

Scoring is **open-ended and all or nothing**. There is no per-case rubric and no partial credit: you either demonstrate the two things below, and receive the full 20 points, or you do not, and receive none.

1. Correctness — the models really run under HE on the TPU. Both LeNet and AlexNetTiny must execute end to end on encrypted inputs, on the accelerator, and the decrypted predictions must match the same model evaluated in cleartext on the same images. Show the comparison, not just a plausible-looking label. An `argmax` that happens to be right is not evidence; the judge will look for per-image agreement against your own cleartext run, and for a server process that never sees the secret key or a plaintext activation.

2. Performance — at least competitive with the state of the art. Measure against the best open-source encrypted-CNN implementation the organizers name for these models, on the same images and batch sizes. Your submission qualifies when it is **no more than 50% slower**, or faster:

```
T <= 1.5 * T_sota
```

for the measured runtime `T` against the state-of-the-art runtime `T_sota` — equivalently, your throughput must reach at least two thirds of theirs. Publish both numbers, the hardware each ran on, and how you established `T_sota`. A comparison across different hardware is not by itself a speedup claim, and you should say so where it applies.

Both conditions are required. A correct implementation that is three times slower than the state of the art earns zero bonus points, and so does a fast one whose correctness cannot be shown. The judge assesses the demonstration on its merits rather than by arithmetic, so make the evidence easy to check: raw timings, the cleartext comparison, the parameter set with `log2(QP)` and its margin, and the commands that reproduce all of it.

Submit `src/`, `cross.patch`, `REPORT.md`, raw result JSON and environment/ weight hashes. Include both successful and failed configurations in the report. Four report pages are sufficient; logs and code are additional. Do not submit datasets, trained weights, secret keys, caches, generated reference outputs or unrelated files. `make submit` packages the listed submission files; it does not upload them.

---

5. References

1. Tong et al., Leveraging ASIC AI Chips for Homomorphic Encryption, HPCA 2026; supplied `CROSS_paper.pdf` and paper. Use BAT/MAT and profiling ideas; paper numbers are not contest baselines.
2. CROSS tutorial: CROSS, TPU Novice to Master, NTT Algorithms and Open Challenge.
3. CROSS `jaxite_word/API_REFERENCE.md`, `nn.py`, `packing.py`, `mapping.py`, `bsgs.py`, `profiler.py`; `profile_analysis/analyze_trace_json.py`.
4. CROSS `demos/README.md`, `lenet_train.py`, `alexnet_infer.py`, `canonical_demo.py`, `encrypted_demos.py`, `lenet_he_test.py` and `alexnet_he_test.py`. Current canonical code takes precedence over old insecure-ring or pre-unification performance tables.